ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA **TALLER 3:** **Análisis en PlantUML**	Departamento: Departamento de Ciencias de la Computación Carrera: Ingeniería de Software Asignatura: Análisis y Diseño de Software NRC: 30723 Fecha: 04/05/2026 Integrantes Grupo 3: • Ayuquina Danny. • Cañarte Saray. • Villagómez Doménica.

DIAGRAMAS DE CASOS DE USO DE ANÁLISIS EN PLANTUML

1. Requisitos Funcionales

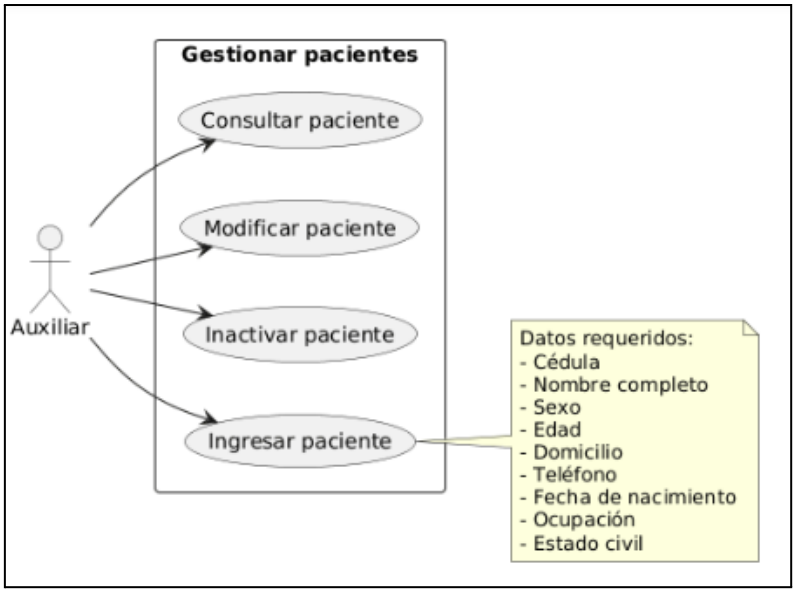
Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir registrar un paciente con cédula, nombre completo, sexo, edad, domicilio, teléfono, fecha de nacimiento, ocupación y estado civil.	Auxiliar	Ingresar paciente
RF-02	El sistema debe permitir actualizar la información de un paciente existente con validación de datos.	Auxiliar	Modificar paciente
RF-03	El sistema debe permitir inactivar un paciente siempre que no tenga citas futuras programadas, registrando el motivo de inactivación.	Auxiliar	Inactivar paciente
RF-04	El sistema debe permitir consultar pacientes registrados mediante listado, búsqueda y visualización de detalles.	Auxiliar	Consultar paciente

2. Diagramas PlantUML

2.1. Casos de Uso

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false
actor "Auxiliar" as Aux

rectangle "Gestionar pacientes" {
  usecase "Ingresar paciente" as UC1
  usecase "Inactivar paciente" as UC2
  usecase "Modificar paciente" as UC3
```

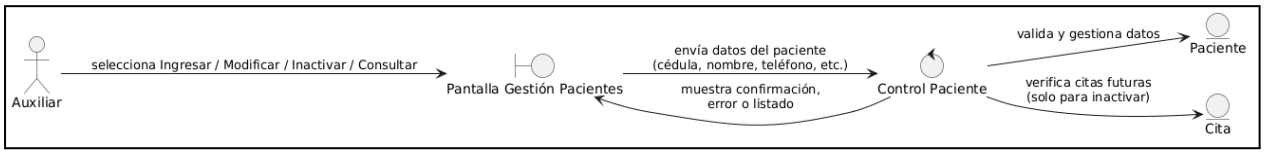
```
  usecase "Consultar paciente" as UC4
}

Aux --> UC1
Aux --> UC2
Aux --> UC3
Aux --> UC4

note right of UC1
Datos requeridos:
- Cédula
- Nombre completo
- Sexo
- Edad
- Domicilio
- Teléfono
- Fecha de nacimiento
- Ocupación
- Estado civil
end note
@enduml
```



2.2. Puente

```
@startuml
left to right direction
actor Auxiliar
boundary "Pantalla Gestión Pacientes" as UI
control "Control Paciente" as Ctrl
entity "Paciente" as Pac
entity "Cita" as Cita

Auxiliar --> UI : selecciona Ingresar / Modificar / Inactivar / Consultar
UI --> Ctrl : envía datos del paciente\n(cédula, nombre, teléfono, etc.)
```

```
Ctrl --> Pac : valida y gestiona datos
Ctrl --> Cita : verifica citas futuras\n(solo para inactivar)
Ctrl --> UI : muestra confirmación,\nerror o listado
@enduml
```

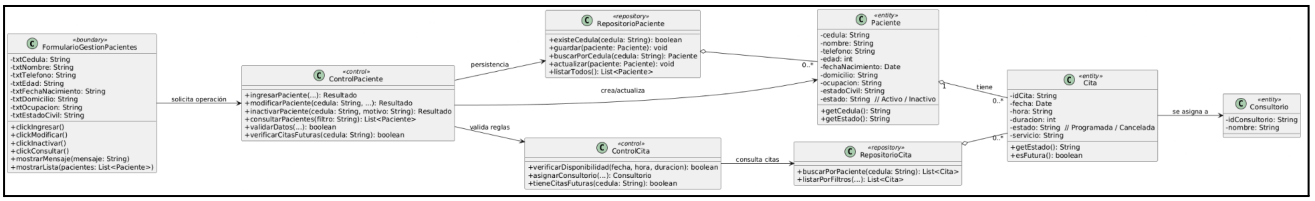


2.3. Clases de Análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "FormularioGestionPacientes" <<boundary>> {
  - txtCedula: String
  - txtNombre: String
  - txtTelefono: String
  - txtEdad: String
  - txtFechaNacimiento: String
  - txtDomicilio: String
  - txtOcupacion: String
  - txtEstadoCivil: String
  + clickIngresar()
  + clickModificar()
  + clickInactivar()
  + clickConsultar()
  + mostrarMensaje(mensaje: String)
  + mostrarLista(pacientes: List<Paciente>)
}

class "ControlPaciente" <<control>> {
  + ingresarPaciente(...): Resultado
  + modificarPaciente(cedula: String, ...): Resultado
  + inactivarPaciente(cedula: String, motivo: String): Resultado
  + consultarPacientes(filtro: String): List<Paciente>
  + validarDatos(...): boolean
  + verificarCitasFuturas(cedula: String): boolean
}

class "Paciente" <<entity>> {
  - cedula: String
  - nombre: String
  - telefono: String
```

```
  - edad: int
  - fechaNacimiento: Date
  - domicilio: String
  - ocupacion: String
  - estadoCivil: String
  - estado: String  // Activo / Inactivo
  + getCedula(): String
  + getEstado(): String
}

class "Cita" <<entity>> {
  - idCita: String
  - fecha: Date
  - hora: String
  - duracion: int
  - estado: String  // Programada / Cancelada
  - servicio: String
  + getEstado(): String
  + esFutura(): boolean
}

class "Consultorio" <<entity>> {
  - idConsultorio: String
  - nombre: String
}

class "ControlCita" <<control>> {
  + verificarDisponibilidad(fecha, hora, duracion): boolean
  + asignarConsultorio(...): Consultorio
  + tieneCitasFuturas(cedula: String): boolean
}

class "RepositorioPaciente" <<repository>> {
  + existeCedula(cedula: String): boolean
  + guardar(paciente: Paciente): void
  + buscarPorCedula(cedula: String): Paciente
  + actualizar(paciente: Paciente): void
  + listarTodos(): List<Paciente>
}

class "RepositorioCita" <<repository>> {
  + buscarPorPaciente(cedula: String): List<Cita>
  + listarPorFiltros(...): List<Cita>
}
```

```
FormularioGestionPacientes --> ControlPaciente : solicita operación
ControlPaciente --> Paciente : crea/actualiza
ControlPaciente --> RepositorioPaciente : persistencia
ControlPaciente --> ControlCita : valida reglas
ControlCita --> RepositorioCita : consulta citas
Paciente "1" o-- "0..*" Cita : tiene
Cita --> Consultorio : se asigna a
RepositorioPaciente o-- "0..*" Paciente
RepositorioCita o-- "0..*" Cita
@enduml
```

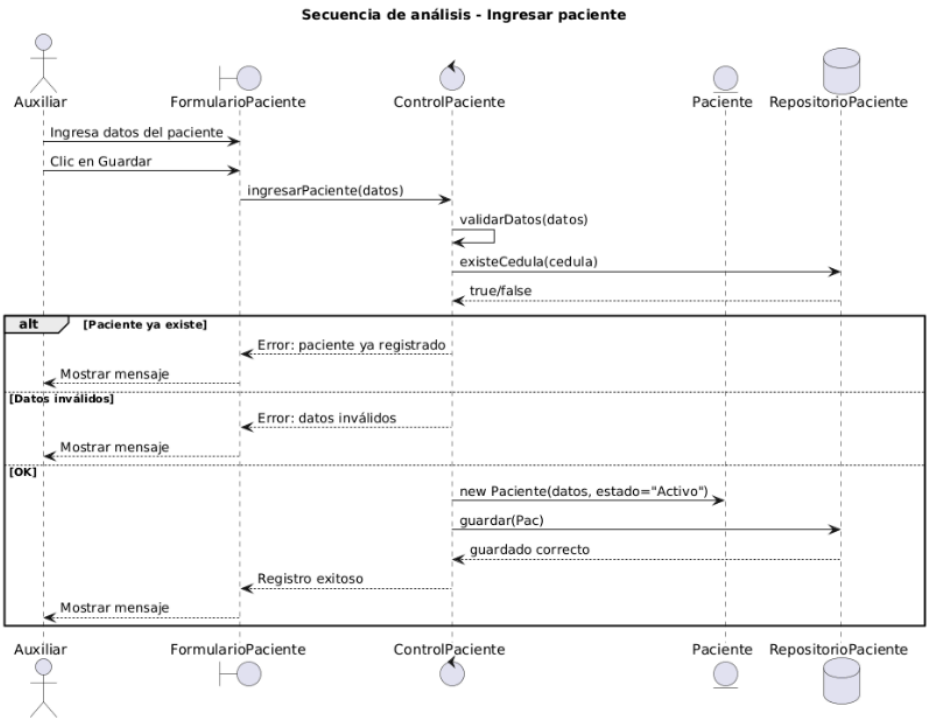


2.4. Secuencia

2.4.1. Ingresar paciente

```
@startuml
title Secuencia de análisis - Ingresar paciente
actor "Auxiliar" as Aux
boundary "FormularioPaciente" as UI
control "ControlPaciente" as Ctrl
entity "Paciente" as Pac
database "RepositorioPaciente" as Repo
Aux -> UI : Ingresa datos del paciente
Aux -> UI : Clic en Guardar
UI -> Ctrl : ingresarPaciente(datos)
Ctrl -> Ctrl : validarDatos(datos)
Ctrl -> Repo : existeCedula(cedula)
Repo --> Ctrl : true/false
alt Paciente ya existe
  Ctrl --> UI : Error: paciente ya registrado
  UI --> Aux : Mostrar mensaje
else Datos inválidos
  Ctrl --> UI : Error: datos inválidos
  UI --> Aux : Mostrar mensaje
else OK
  Ctrl -> Pac : new Paciente(datos, estado="Activo")
  Ctrl -> Repo : guardar(Pac)
  Repo --> Ctrl : guardado correcto
  Ctrl --> UI : Registro exitoso
  UI --> Aux : Mostrar mensaje
end
```

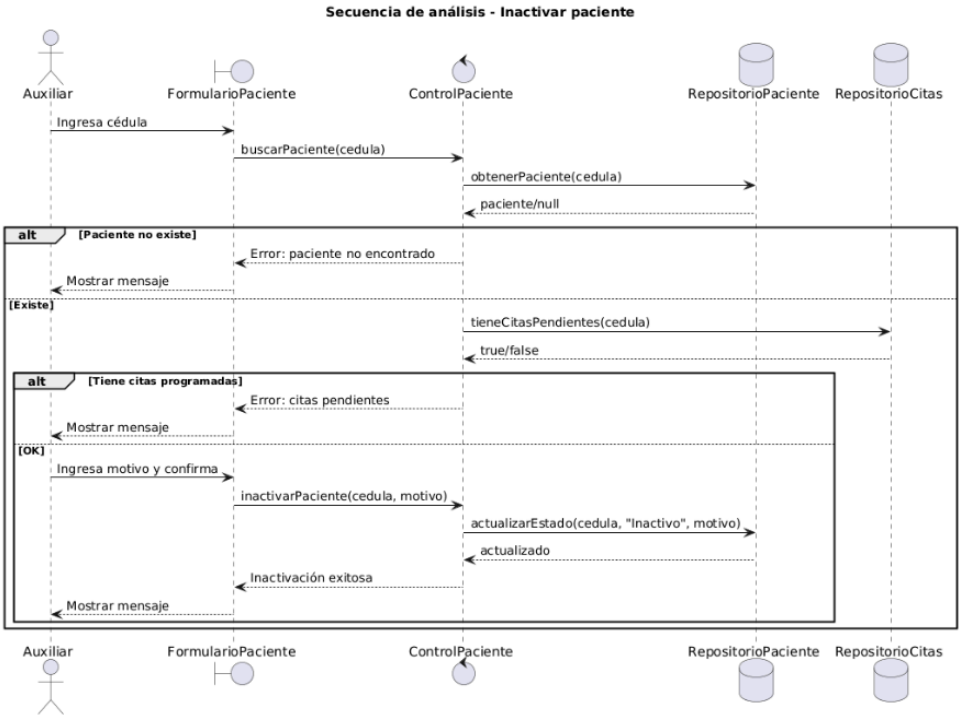
```
@enduml
```



2.4.2. Inactivar Paciente

```
@startuml
title Secuencia de análisis - Inactivar paciente
actor "Auxiliar" as Aux
boundary "FormularioPaciente" as UI
control "ControlPaciente" as Ctrl
database "RepositorioPaciente" as Repo
database "RepositorioCitas" as RepoCitas
Aux -> UI : Ingresa cédula
UI -> Ctrl : buscarPaciente(cedula)
Ctrl -> Repo : obtenerPaciente(cedula)
Repo --> Ctrl : paciente/null
alt Paciente no existe
  Ctrl --> UI : Error: paciente no encontrado
  UI --> Aux : Mostrar mensaje
else Existe
  Ctrl -> RepoCitas : tieneCitasPendientes(cedula)
  RepoCitas --> Ctrl : true/false
  alt Tiene citas programadas
    Ctrl --> UI : Error: citas pendientes
    UI --> Aux : Mostrar mensaje
  else OK
    Aux -> UI : Ingresa motivo y confirma
    UI -> Ctrl : inactivarPaciente(cedula, motivo)
```

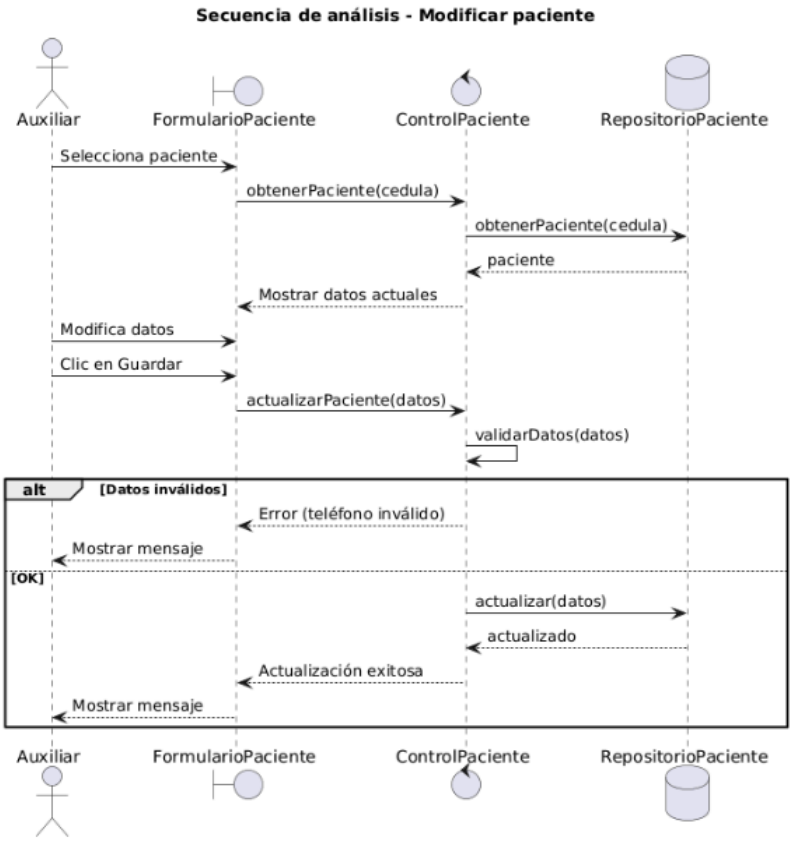
```
  Ctrl -> Repo : actualizarEstado(cedula, "Inactivo", motivo)
  Repo --> Ctrl : actualizado
  Ctrl --> UI : Inactivación exitosa
  UI --> Aux : Mostrar mensaje
 end
end
@enduml
```



2.4.3. Modificar Paciente

```
@startuml
title Secuencia de análisis - Modificar paciente
actor "Auxiliar" as Aux
boundary "FormularioPaciente" as UI
control "ControlPaciente" as Ctrl
database "RepositorioPaciente" as Repo
Aux -> UI : Selecciona paciente
UI -> Ctrl : obtenerPaciente(cedula)
Ctrl -> Repo : obtenerPaciente(cedula)
Repo --> Ctrl : paciente
Ctrl --> UI : Mostrar datos actuales
Aux -> UI : Modifica datos
Aux -> UI : Clic en Guardar
UI -> Ctrl : actualizarPaciente(datos)
Ctrl -> Ctrl : validarDatos(datos)
alt Datos inválidos
  Ctrl --> UI : Error (teléfono inválido)
```

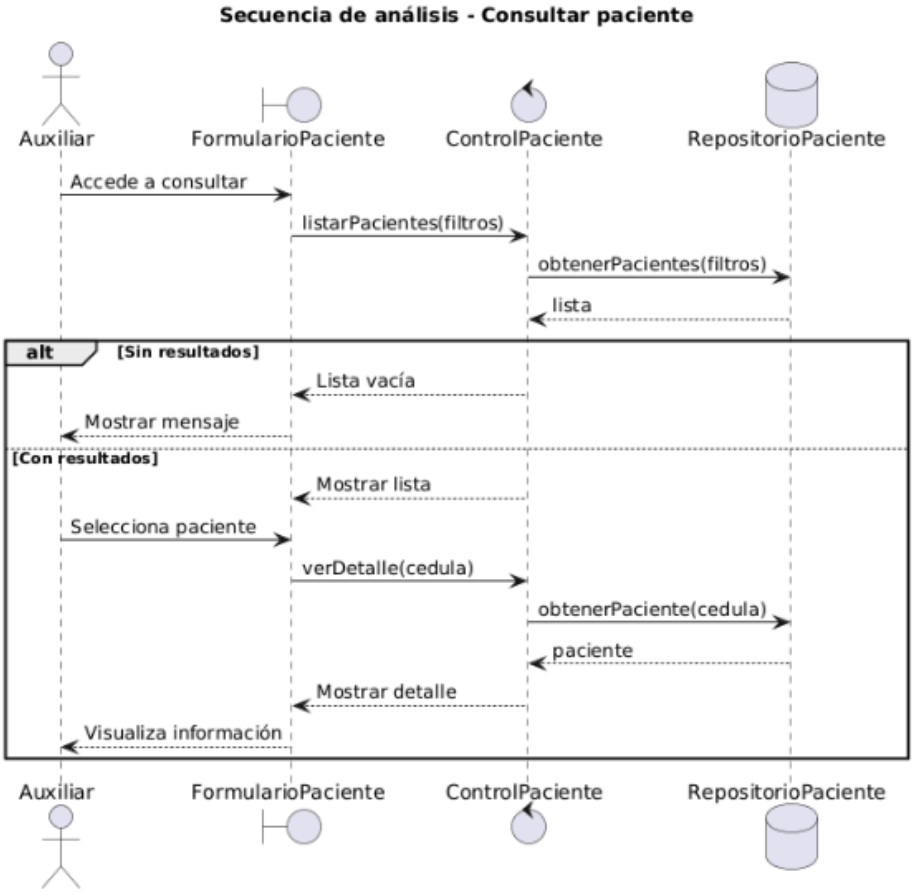
```
  UI --> Aux : Mostrar mensaje
else OK
  Ctrl -> Repo : actualizar(datos)
  Repo --> Ctrl : actualizado
  Ctrl --> UI : Actualización exitosa
  UI --> Aux : Mostrar mensaje
end
@enduml
```



2.4.4. Consultar Paciente

```
@startuml
title Secuencia de análisis - Consultar paciente
actor "Auxiliar" as Aux
boundary "FormularioPaciente" as UI
control "ControlPaciente" as Ctrl
database "RepositorioPaciente" as Repo
Aux -> UI : Accede a consultar
UI -> Ctrl : listarPacientes(filtros)
Ctrl -> Repo : obtenerPacientes(filtros)
Repo --> Ctrl : lista
alt Sin resultados
```

```
 Ctrl --> UI : Lista vacía
 UI --> Aux : Mostrar mensaje
else Con resultados
 Ctrl --> UI : Mostrar lista
 Aux -> UI : Selecciona paciente
 UI -> Ctrl : verDetalle(cedula)
 Ctrl -> Repo : obtenerPaciente(cedula)
 Repo --> Ctrl : paciente
 Ctrl --> UI : Mostrar detalle
 UI --> Aux : Visualiza información
end
@enduml
```

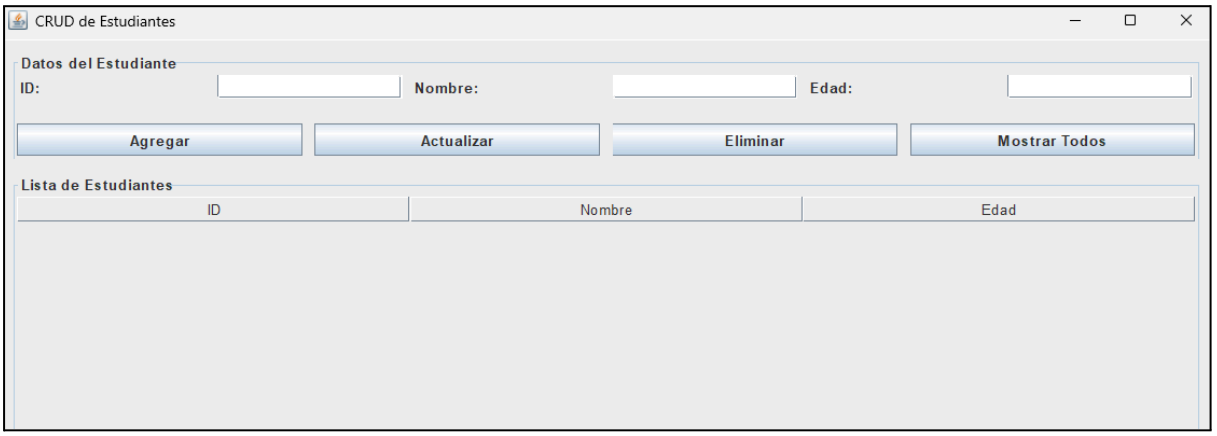


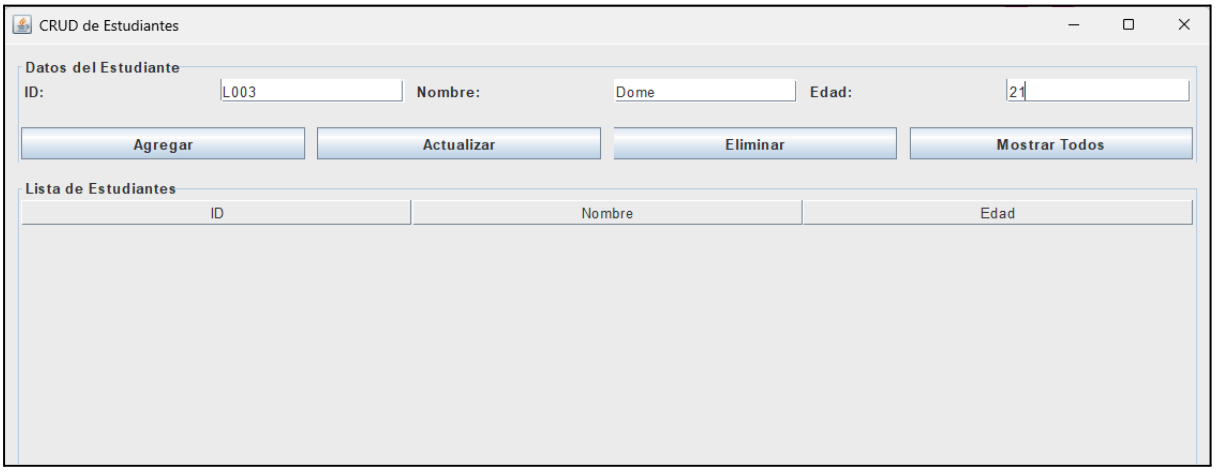
3. Matriz Trazabilidad

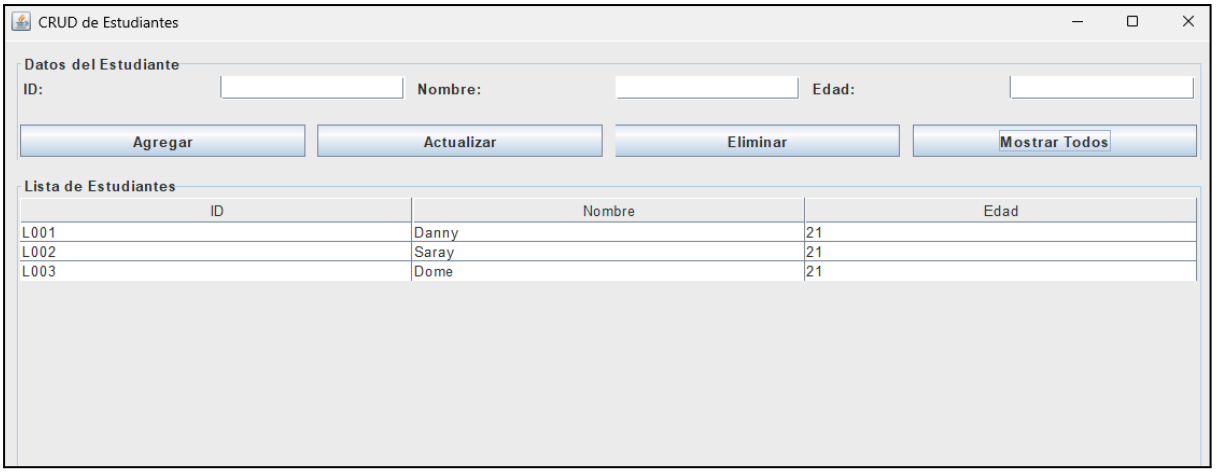
RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Ingresar Paciente	Auxiliar	Alta	El sistema guarda un paciente con: cédula, nombre completo, sexo, edad, domicilio, teléfono,	UC1

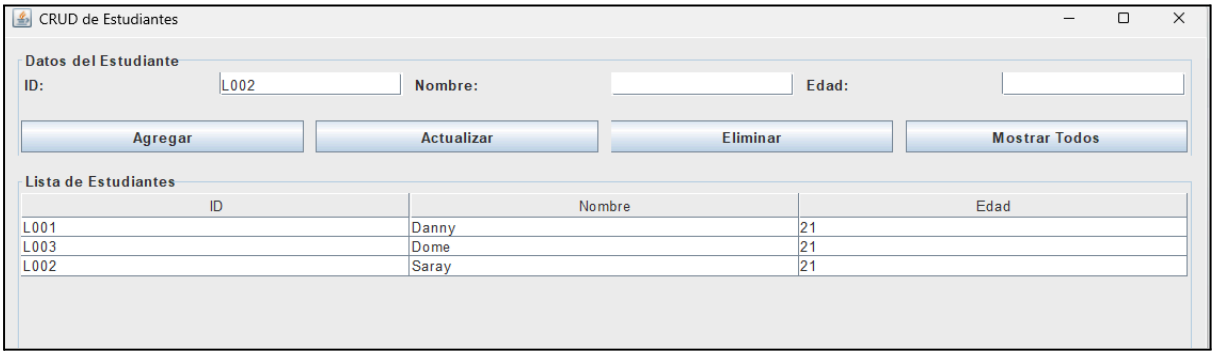
				fecha de nacimiento, ocupación y estado civi .	
RF-02	Inactivar Paciente	Auxiliar	Alta	El sistema marca como eliminado al paciente seleccionado.	UC2
RF-03	Modificar Paciente	Auxiliar	Media	El sistema permite modificar cualquiera de los campos antes mencionados para la corrección o actualización de datos.	UC3
RF-04	Consultar Paciente	Auxiliar	Alta	El sistema muestra el listado de pacientes y permite búsqueda/filtros, con el fin de poder escoger un paciente y mostrar los detalles de su información.	UC4

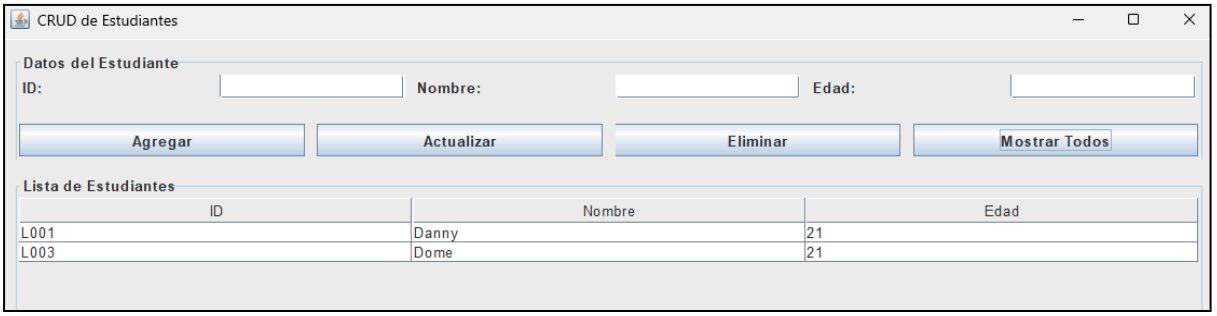
4. Implementación de CRUD de Estudiante











5. Análisis y conclusiones

La coherencia entre el diagrama de secuencia, el diagrama de clases, el puente hacia clases de análisis y los casos de uso es fundamental para garantizar que un sistema de gestión de estudiantes funcione correctamente y pueda ser implementado. Cada uno de estos elementos cumple un rol distinto, pero deben estar alineados para representar de forma consistente cómo el sistema agrega, actualiza, elimina y muestra estudiantes.

Los casos de uso definen claramente qué acciones debe realizar el sistema, mientras que el diagrama de secuencia muestra paso a paso cómo interactúan los objetos para cumplir esas acciones. Por su parte, el diagrama de clases establece la estructura del sistema, definiendo atributos y métodos que deben coincidir con las operaciones descritas. El uso del puente hacia clases de análisis permite identificar clases de frontera, control y entidad, de manera que simplifica la comprensión entre la interacción, la verificación y conservación de datos del sistema.

Cuando existe coherencia entre estos elementos y el código final, se logra un sistema más claro, organizado y fácil de mantener. Además, se reducen errores, se mejora la trazabilidad entre el diseño y la implementación, y se facilita la escalabilidad del sistema ante nuevos requerimientos.